```
          image: catalog-service
          imagePullPolicy: IfNotPresent
          lifecycle:
            preStop:                    ❶
              exec:
                command: [ "sh", "-c", "sleep 5" ]
          ...
```

❶ Makes Kubernetes wait 5 seconds before sending the SIGTERM signal to the Pod

Finally, apply the updated version of the Deployment object with `kubectl apply -f k8s/deployment.yml`. Kubernetes will reconcile the new desired state and replace the existing Pod with a new one for which graceful shutdown is fully configured.

NOTE When a Pod contains multiple containers, the `SIGTERM` signal is sent to all of them in parallel. Kubernetes will wait up to 30 seconds. If any of the containers in the Pod are not terminated yet, it will shut them down forcefully.

Now that we've configured the graceful shutdown behavior for Catalog Service, let's look at how to scale it in a Kubernetes cluster.

7.4.3 Scaling Spring Boot applications

Scalability is one of the main properties of a cloud native application, as you learned in chapter 1. To be scalable, applications should be disposable and stateless, as per the 15-Factor methodology.

We handled disposability in the previous section, and Catalog Service is already a stateless application. It has no state but relies on a stateful service (the PostgreSQL database) to permanently store the data about books. We scale applications in and out, and if they weren't stateless, we would lose the state every time an instance is shut down. The general idea is to keep the applications stateless and rely on data services for storing the state, just like we do in Catalog Service.